

RELATÓRIO TEÓRICO

Jogo de Plataforma 2D na Unity

Tema do Jogo: Aventura Urbana — Streets of Code

Disciplina: Game Development | UniFECAF

Este relatório apresenta os fundamentos teóricos e práticos aplicados no desenvolvimento do jogo de plataforma 2D *Streets of Code*, criado com a Unity Engine e programado em C#. O documento abrange desde os princípios do gênero até as decisões de design, programação e experiência do usuário.

1. Princípios dos Jogos de Plataforma 2D

Os jogos de plataforma 2D constituem um dos gêneros mais clássicos e influentes da história dos videogames. Surgidos nos anos 1980 com títulos como Donkey Kong (1981) e consagrados por Super Mario Bros. (1985), esses jogos são caracterizados pela navegação de um personagem por cenários compostos de plataformas, obstáculos e elementos coletáveis, geralmente em uma perspectiva lateral.

Os pilares fundamentais do gênero são:

Controle responsivo: O jogador deve sentir que o personagem responde imediatamente às suas ações. Qualquer atraso ou imprecisão nos controles é percebido como falha de design.

Progressão de dificuldade: Os níveis devem ensinar mecânicas gradualmente, começando simples e aumentando o desafio de forma justa. O conceito de 'Nintendo Hard' define o equilíbrio entre desafio e satisfação.

Feedback imediato: Sons, animações e efeitos visuais devem confirmar cada ação do jogador — pulo, coleta, dano — reforçando a sensação de imersão.

Level design intuitivo: O cenário deve guiar o jogador naturalmente, usando elementos visuais para sinalizar caminhos, perigos e recompensas sem necessidade de tutoriais extensos.

2. Planejamento do Projeto e Mecânicas Propostas

Streets of Code é um jogo de plataforma 2D ambientado em uma metrópole futurista e decadente. O jogador controla um jovem hacker que precisa atravessar 4 fases urbanas — ruas, becos, telhados e metrô — coletando chips de dados e evitando drones de segurança.

Mecânicas implementadas:

Movimento horizontal: Andar e correr com velocidades distintas, controladas pelo eixo horizontal do Input System da Unity.

Pulo: Implementado com força aplicada ao Rigidbody2D. Um segundo pulo no ar (double jump) é desbloqueado no nível 2.

Escalada: O personagem pode escalar escadas e grades usando detecção de trigger e desativando temporariamente a gravidade.

Coleta de itens: Chips de dados espalhados pelos níveis aumentam a pontuação. Itens especiais concedem vida extra.

Sistema de vidas: O jogador inicia com 3 vidas. Ao contato com inimigos ou queda em zonas de perigo, perde uma vida.

3. Design de Níveis com Progressão de Dificuldade

O design de níveis segue o princípio de progressão orgânica: cada fase apresenta novos desafios que exigem o domínio das mecânicas ensinadas na fase anterior.

Nível 1 — Ruas da Cidade: Fase de introdução. Plataformas estáveis, obstáculos fixos e inimigos parados. Ensina movimento básico e coleta.

Nível 2 — Becos e Vias: Introduz plataformas móveis e o double jump. Inimigos com patrulha simples. Tempo começa a ser fator.

Nível 3 — Telhados: Plataformas instáveis que desaparecem após o toque. Ventos laterais afetam o movimento. Inimigos com projéteis.

Nível 4 — Metrô Subterrâneo: Fase final. Combina todas as mecânicas. Trens como obstáculos móveis, plataformas, inimigos e chefe final.

4. Interatividade, HUD e Feedback

A interface HUD (Heads-Up Display) foi projetada para ser minimalista e funcional, exibindo apenas as informações essenciais sem poluir visualmente o cenário urbano.

Elementos do HUD:

Pontuação: Exibida no canto superior direito, atualizada em tempo real ao coletar chips.

Vidas: Ícones de corações no canto superior esquerdo. Piscam em vermelho ao tomar dano.

Indicador de fase: Nome da fase exibido brevemente ao iniciar cada nível.

Sistema de Feedback:

Visual: Partículas ao coletar itens, flash vermelho ao tomar dano, animação de morte.

Sonoro: Som distinto para pulo, coleta, dano e morte. Trilha sonora ambiente por fase.

Câmera: Leve shake de câmera ao tomar dano, reforçando o impacto para o jogador.

5. Uso de Assets, Sprites e Animações na Unity

Os sprites utilizados foram obtidos do repositório gratuito itch.io e do Unity Asset Store (licença gratuita), com estética pixel art que remete ao visual cyberpunk urbano.

Pipeline de animação na Unity:

1. Importação dos spritesheets como Texture Type: Sprite (2D and UI), com Sprite Mode: Multiple para fatiamento automático.

2. Criação de Animation Clips para cada estado: Idle, Run, Jump, Fall, Climb, Hurt e Death.
3. Configuração do Animator Controller com parâmetros booleanos e float (Speed, IsGrounded, IsHurt) para controlar as transições entre estados automaticamente.
4. Script PlayerAnimator em C# lê os valores do PlayerController e os passa ao Animator via SetBool() e SetFloat().

6. Lógica de Programação em C# e Estrutura do Game Loop

A arquitetura de scripts segue o padrão de responsabilidade única: cada script gerencia um aspecto específico do jogo, facilitando manutenção e expansão.

Estrutura principal de scripts:

PlayerController.cs: Gerencia input, movimento, pulo e escalada do personagem.

PlayerHealth.cs: Controla vidas, dano, invencibilidade temporária e morte.

CollectibleItem.cs: Detecta colisão com o jogador via OnTriggerEnter2D e adiciona pontos.

EnemyPatrol.cs: Movimenta inimigos entre pontos de patrulha com detecção de borda.

GameManager.cs: Singleton que gerencia pontuação, vidas, transição de cenas e estado do jogo.

UIManager.cs: Atualiza os elementos do HUD conforme eventos do GameManager.

LevelLoader.cs: Gerencia transições entre fases com Coroutine e fade de tela.

Estrutura do Game Loop:

O loop de jogo na Unity é gerenciado automaticamente pela engine. O método **Update()** processa input e lógica a cada frame, enquanto **FixedUpdate()** trata a física do Rigidbody2D em intervalos fixos (0.02s), garantindo comportamento consistente independente do framerate da máquina.

7. Estratégias de Inserção de Áudio e Efeitos Sonoros

O design de áudio foi pensado para reforçar a atmosfera urbana futurista e fornecer feedback auditivo claro ao jogador.

Implementação técnica:

AudioManager (Singleton): Script central que armazena referências a todos os AudioClips e expõe métodos PlaySFX() e PlayMusic() acessíveis por qualquer script do jogo.

AudioSource dedicado para música: Loop contínuo com volume reduzido (0.4f) para não sobrepor os efeitos sonoros.

Efeitos sonoros por evento: Pulo, aterrissagem, coleta de chip, dano recebido, morte e transição de fase possuem AudioClips distintos, chamados via `AudioManager.Instance.PlaySFX()`.

Assets de áudio: Trilha sonora Lo-fi synthwave e efeitos 8-bit obtidos do freesound.org (licença Creative Commons).

8. Conexão entre Teoria e Prática

O desenvolvimento de Streets of Code evidenciou como os conceitos teóricos do game design se manifestam em decisões práticas concretas durante a produção:

Curva de aprendizado: A progressão de 4 fases foi ajustada iterativamente durante os testes, removendo obstáculos que frustravam sem ensinar.

Agência do jogador: O double jump e a escalada opcionais permitem ao jogador escolher rotas diferentes nos níveis 2 e 3.

Narrativa ambiental: Cartazes, janelas quebradas e neon lights no cenário contam a história da cidade sem diálogos.

Loop de gameplay: A estrutura explorar → coletar → superar obstáculo → avançar repete-se satisfatoriamente em todos os níveis.

Feedback e imersão: Cada elemento de HUD e som foi testado para confirmar ações sem distrair o jogador do fluxo de jogo.

Este projeto demonstrou que o game design eficaz nasce da integração entre teoria sólida e iteração prática constante. Cada decisão técnica — da escolha do RigidBody2D ao design dos níveis — refletiu princípios estudados na disciplina, resultando em uma experiência jogável coesa e funcional.